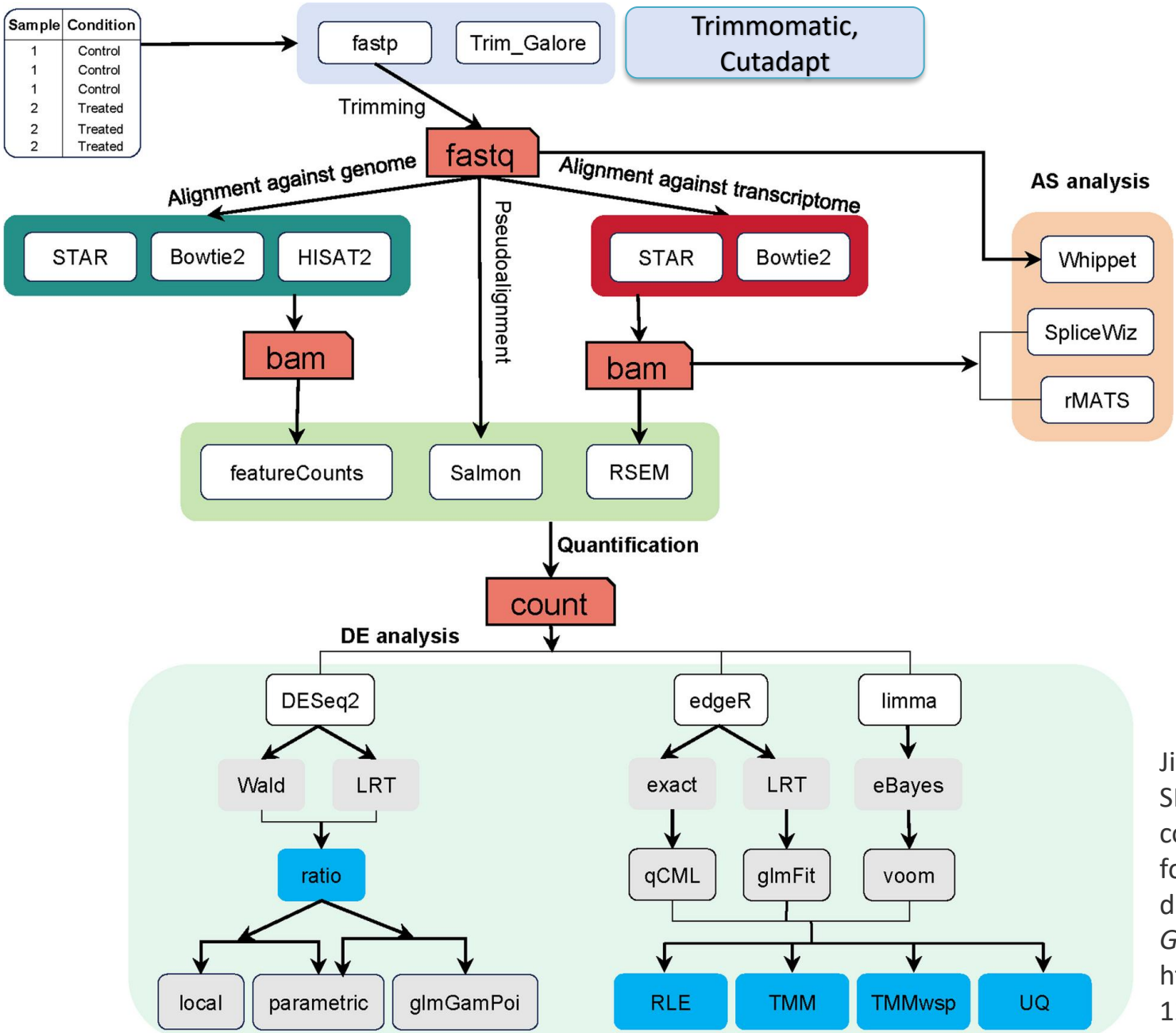
UNIVERSITY OF PISA

Lesson 9

Silvia Giulia Galfrè

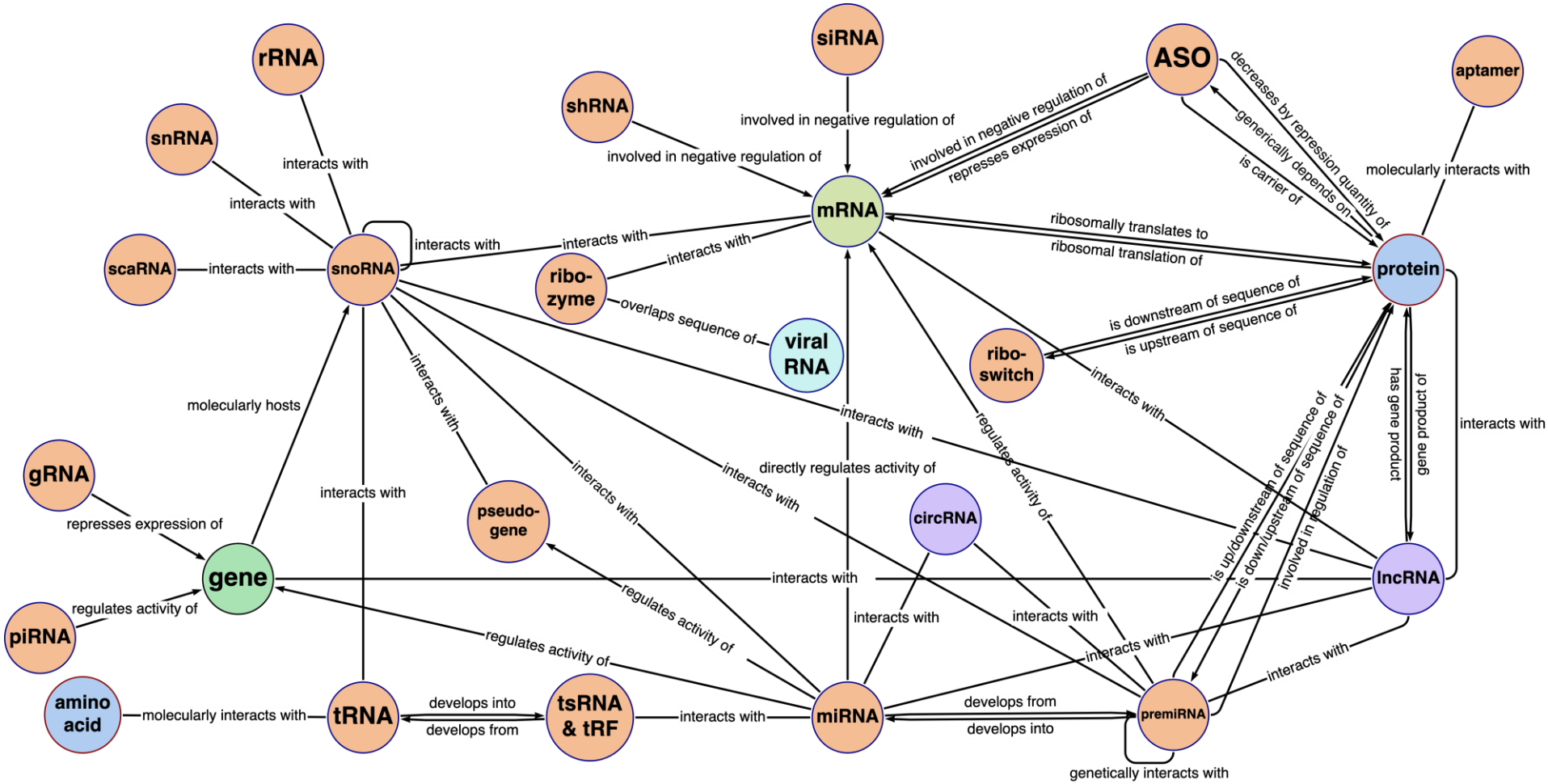


Jiang, G., Zheng, JY., Ren, SN. *et al.* A comprehensive workflow for optimizing RNA-seq data analysis. *BMC Genomics* **25**, 631 (2024). https://doi.org/10.1186/s12864-024-10414-y

Rna seq

- Main aims:
  - Gene expression
  - New transcript identification
  - Fusion transcript detection

The RNAs



Cavalleri, E. et al. An ontology-based knowledge graph for representing interactions involving RNA molecules. Sci Data 11, 906 (2024). https://doi.org/10.1038/s41597-024-03673-7

DEG analysis (or DEA)

- The adopted pipeline significantly affects the DEG output
  - No single tool is optimal il all circumstances
  - Especially the number of replicates and heterogeneity of samples should be considered in selecting the tools

- Some important issues regarding its impact on the results.
  - access to the reference genome or transcriptome,
  - quality of annotations
  - The number of samples.

Methods division

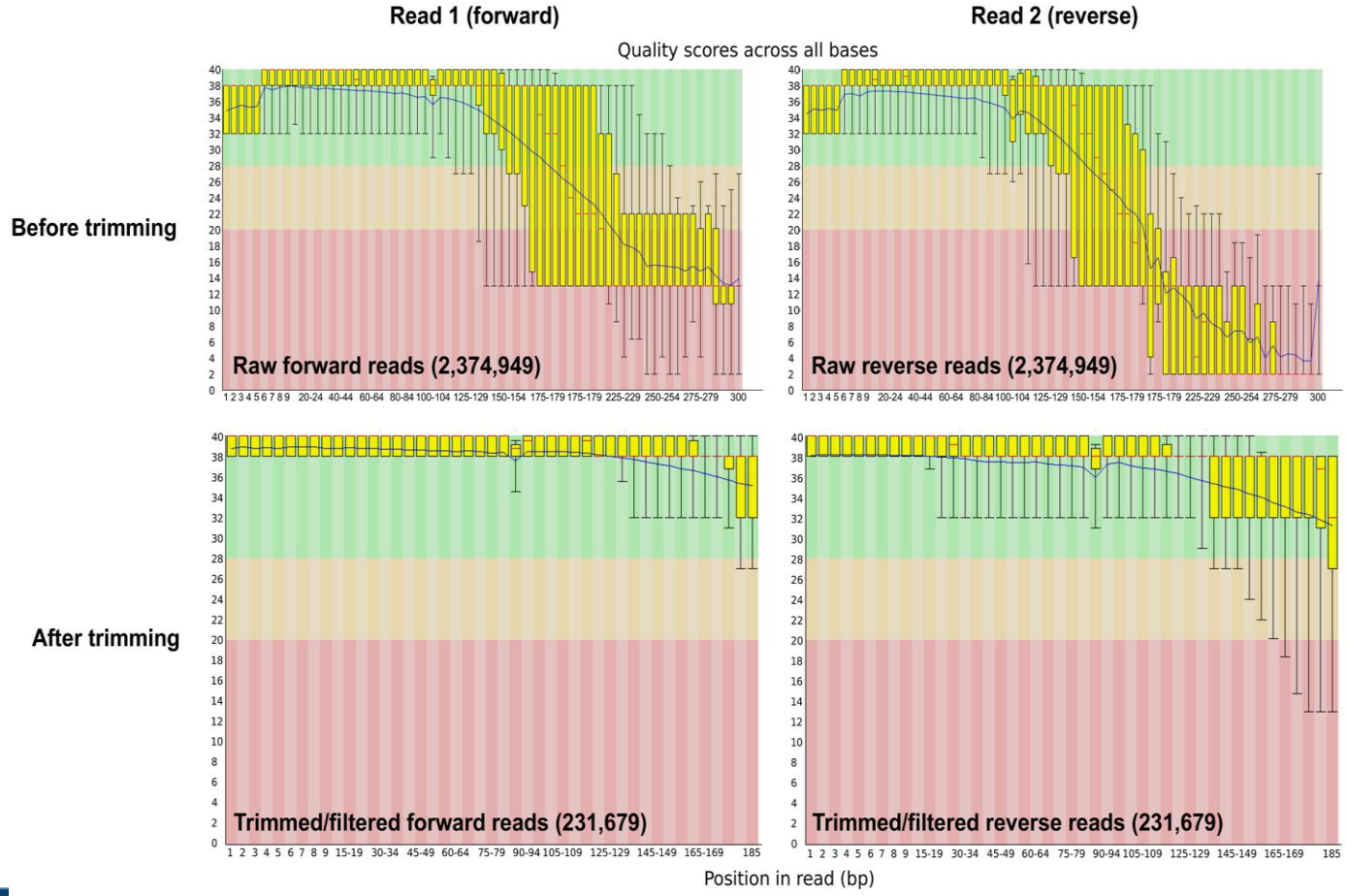
- Methods for DEG analysis can be divided by how they treat the expression distribution.
  - Approaches that consider a specific distribution for the data: parametric approaches.
  - Methods that do not consider data distribution: non-parametric approaches.
  - An alternative is to consider both or more approaches: hybrid approaches

Step1: Quality assessment and sequence trimming

- common to other analyses involving sequencing data, such as genome and transcript assemblies
- Quality assessment aims to identify and remove sequences identified as having low quality.
  - the quality information in the FASTQ file is used.
- Trimming removes sequences from adapters used in sequencing.
  - performed by similarity search

FASTQC

- generates quality reports and does not perform the removal of low-quality sequences or adapters.



Sequence Quality Per-base Before and After Trimming and Adapter Removal.

Tennant, R. K., Sambles, C. M., Diffey, G. E., Moore, K. A., Love, J. Metagenomic Analysis of Silage. J. Vis. Exp. (119), e54936, doi:10.3791/54936 (2017).

Removal of low-quality sequences or adapters tools

Name	Year	Reference
Btrim	2011	Paper
CutAdapt	2011	Paper site
Trimmomatic	2014	Paper site
AdapterRemoval v2	2016	Paper site
Atropos	2017	Paper site
fastp	2018	Paper site

Cutadapt

- **Strengths**
  - **Extremely flexible adapter handling**
    Supports 5'/3' adapters, anchored adapters, linked adapters, variable-length adapters, overlapping modes, etc. cutadapt.readthedocs.io+1
  - **High trimming accuracy**
    Benchmarks generally show very good sensitivity for removing true adapter sequence. PMC+1
  - **Mature & well documented**
    Long history, excellent docs, widely used, lots of community examples. cutadapt.readthedocs.io+1
- **Weaknesses**
  - **Not the fastest**
    Historically slower than newer C++ tools (e.g. fastp) and often slower than BBDuk/Trimmomatic in speed-focused benchmarks. ResearchGate+1
  - **Narrow scope**
    It focuses on trimming; QC plots, advanced filtering, UMIs, etc. need other tools or extra scripting.

- Best for: **maximum control over adapter trimming logic** and “weird” adapter setups.

Trimmomatic

- **Strengths**
  - **Pipeline-style trimming**
    Combines adapter clipping, sliding-window quality trimming, cropping, length filters, etc., in configurable order. OUP Academic+1
  - **Reasonably fast & multithreaded**
    Written in Java, performs well and has been a workhorse in many RNA-seq/Illumina pipelines. OUP Academic+1
  - **Widely supported**
    Built into lots of wrappers/GUI tools and tutorials.
- **Weaknesses**
  - **Slower & less feature-rich than newer "all-in-one" tools**
    fastp, for example, is usually 2–5× faster while also doing QC and more filtering in a single pass. ResearchGate+1
  - **Adapter logic a bit less flexible**
    Relies on adapter FASTA files and simpler matching; for exotic adapter configurations, Cutadapt/Atropos tend to be more configurable.

- Best for: **legacy / established pipelines** and users who like its simple, modular trimming steps.

AdapterRemoval v2

- **Strengths**
  - **Outstanding for overlapping paired-end reads**
    Very high merge rate and speed; top of the pack for merging in multiple benchmarks, especially useful for short inserts / degraded DNA (e.g. ancient DNA). BioMed Central+2PMC+2
  - **Rich adapter-related features**
    Handles multiple adapter pairs in the same dataset, simultaneous demultiplexing+trimming, and can reconstruct unknown adapters from overlapping pairs. BioMed Central+1
  - **High throughput**
    Uses SIMD and multithreading; among the fastest trimmers in several tests. Research Profiles+1
- **Weaknesses**
  - **Focused tool, fewer extras**
    No built-in HTML QC reports, fancy visualizations, etc. (you'll typically still run FastQC or multiQC separately).
  - **Not always the very fastest for pure trimming**
    For datasets with multiple adapters but no merging, AlienTrimmer and Trimmomatic edged it out in some benchmarks. BioMed Central
- Best for: **short/degraded libraries** and **projects where merging paired-end reads is central**.

Atropos

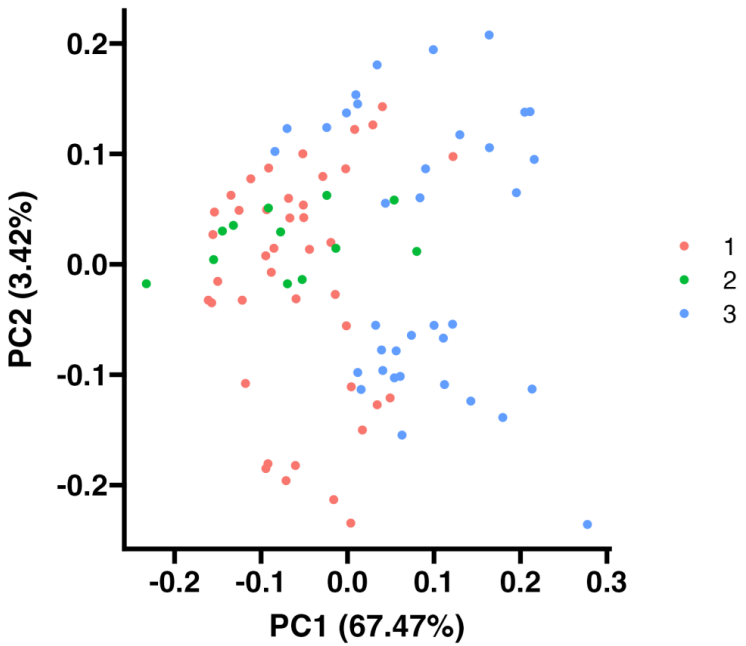
- **Strengths**
  - **High accuracy + speed**
    The Atropos paper reports up to ~4× higher trimming accuracy and ~50% lower runtime than other state-of-the-art trimmers using 16 threads. PMC+1
  - **Cutadapt DNA with extra muscles**
    It's a fork of Cutadapt, so you get its flexible adapter specification plus added features (better multithreading, additional algorithms, error modeling). PMC+1
  - **Advanced options**
    Support for bisulfite data, various quality-trimming strategies, etc., targeting modern high-volume sequencing experiments. PMC+1
- **Weaknesses**
  - **Smaller user base**
    Less "standard" than Trimmomatic/Cutadapt/fastp, so fewer tutorials, wrappers and example pipelines.
  - **Python-based install**
    Heavier environment than a single static binary (like fastp), which can be mildly annoying on some HPC setups.
- Best for: **users who love Cutadapt's flexibility but want better speed and extra advanced trimming modes**.

fastp

- **Strengths**
  - **Very fast**
    C++ with multithreading; typically 2–5× faster than Trimmomatic or Cutadapt in authors' benchmarks, despite doing more work per read. PubMed+2ResearchGate+2
  - **All-in-one preprocessor**
    Adapter trimming (auto-detection or specified), quality filtering, per-read pruning, polyG/polyX trimming, low-complexity filtering, UMI handling, base correction for PE, output splitting, etc., all in a single pass. SciSpace+2Semantic Scholar+2
  - **Built-in QC**
    Generates JSON/HTML reports with per-base quality, content, duplication, etc., so you often don't need a separate FastQC run. BioRxiv+1
  - **Easy deployment**
    Single executable, widely packaged.
- **Weaknesses**
  - **Less "surgical" adapter configuration**
    Cutadapt/Atropos still offer more specialised adapter matching modes; fastp's adapter handling is powerful but more automatic and less fine-grained for edge-case designs. cutadapt.readthedocs.io+1
  - **Heavy reliance on defaults**
    Because it does so much automatically, it's easy to over-trim / over-filter if you don't read the docs and QC report carefully.
- Best for: **fast, convenient "one command" preprocessing + QC**, especially for standard Illumina short-read data.

Step 1 bis: batch effects removal

- An additional point to observe in quality assessment are batch effects.
- Batch effects arise from differences between samples that are not rooted in the experimental design and can have various sources
- Various methods have been developed to detect and/or remove batch effects in genomics data, particularly RNA-seq data.
  - svaseq
  - Combat-Seq



Step 2: Alignment

- Aims to identify how many reads are aligned to a genome region
- Among the difficulties in the mapping process are processing time and the computational capacity required, which have major challenges.

A good way to compress: the Burrows-Wheeler transform

- It's a **reversible rearrangement of a string** that groups similar characters together, which makes the string easier to compress
- Steps:
  - Take a string and append a special end symbol (often $) that is lexicographically smaller than any real character.
  - Form all **cyclic rotations** of that string.
  - Sort these rotations alphabetically.
  - Take the **last column** of this sorted matrix → that's the BWT.
  - You can later reconstruct the original string from just the BWT and the position of $

Example

- Say our DNA string is: GATTA
- As with any BWT, we first append a unique end symbol $: GATTA$
- Rotate the string in all possible ways:

$GA_0T_0T_1A_1\$$

$A_0T_0T_1A_1\$G$

$T_0T_1A_1\$GA_0$

$T_1A_1\$GA_0T_0$

$A_1\$GA_0T_0T_1$

$\$G\ A_0T_0T_1A_1$

Sort

$\$GA_0T_0T_1A_1$

$A_1\$GA_0T_0T_1$

$A_0T_0T_1A_1\$G$

$GA_0T_0T_1A_1\$$

$T_1A_1\$GA_0T_0$

$T_0T_1A_1\$GA_0$

the BWT string

Not only for storing

- The BWT is also very fast to search for a string

Bowtie 2

- Burrows-Wheeler transform.
- Thus, for each read, the method performs four key steps:
  - the extraction of ‘‘seeds” (sequence snippets) from the read and their reverse complement;
  - the alignment of the seeds to the genome (reference), producing Burrows-Wheeler alignment bands;
  - the selection of the bands randomly and repeatedly (weighted by priority), applying the displacement of each selected lane on the reference genome, using a method to compress the suffix matrix, and still effectively support the search for arbitrary patterns, called FM index, and applying the ‘‘walk-left” strategy of the FM index; and
  - the resolution of similar alignments, observing the edges.

STAR

- proposed to specifically address many of the challenges of mapping RNA-seq data, such as junction detection and characterization and mapping sequences derived from non-contiguous genomic regions.
- It uses a novel strategy for junction alignments.
  - The alignment comprises two major steps: seed search and clustering.
- The main idea behind the STAR seed search step is the sequential search for a Maximum Mappable Prefix.
- In the clustering step, STAR builds alignments of the entire read sequence by joining all the seeds that were aligned to the genome in the first step

Tophat 2

- It directs attention to the problem of multiple alignments in junction reads.
- It uses the Bowtie 2 tool as a dependency.
- In the mapping step, reads aligned to over one exon are treated as unmapped. These reads are fragmented and aligned to the genome.
- Tophat 2 considers that the alignment distance between fragments may indicate possible splice regions. The genomic sequences around these junction sites are concatenated, and the resulting spliced sequences are treated as a set of potential transcription fragments.
- Any reads not mapped in the previous stages (or poorly mapped) are then realigned with Bowtie2 against this new transcript.

HISAT2

- It has a graph-based search strategy as its fundamental characteristic.
- It starts the alignment process by generating a linear graph of the reference genome and then it adds mutations, insertions and deletions as alternative paths of the graph.
- The authors claim that graph representations are more efficient in terms of memory utilization and/or alignment speed compared to the linear reference representation of genomes and alleles.

Analyses without a reference genome

- There are a few options:
  - It is possible to generate a transcriptome assembly using the reads from the experiment itself and map it to the generated assembly, using tools such as tools such as RSEM, Cufflinks;
  - Map against the transcriptome, using HISAT2, TopHat and/or kallisto;
  - Map considering several genomes, with tools such as HISAT2.

Salmon pseudoalignment

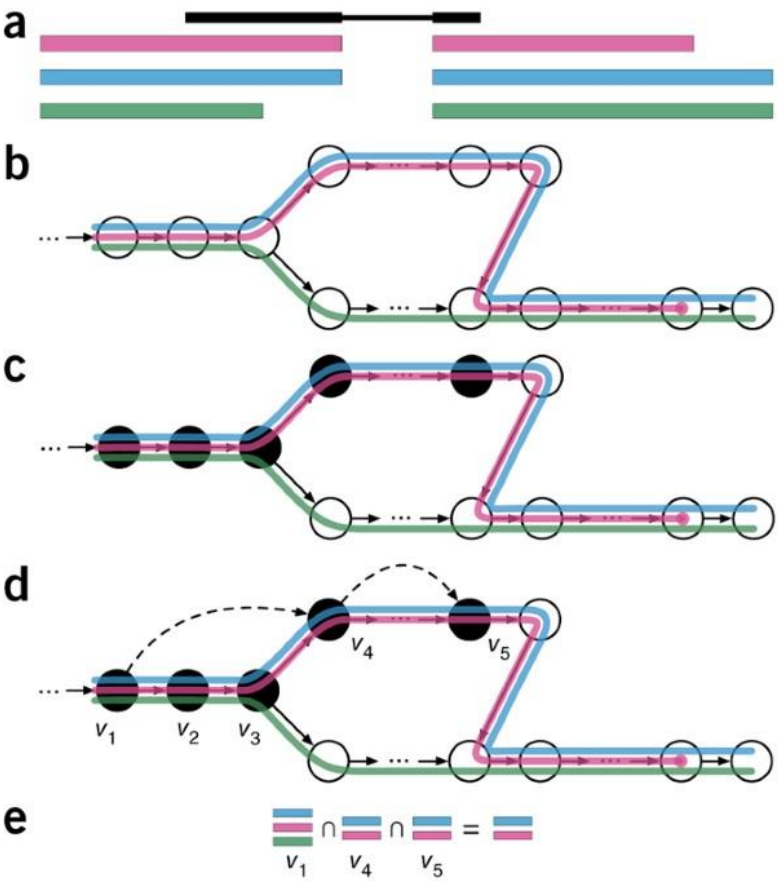
- Salmon uses the quasimapping strategy, which requires a set of transcripts as a reference (which can be a reference assembly or de novo) or only the reads, to quantify the transcripts.
- This strategy comprises three steps
  - a simplified mapping model,
  - a phase that estimates initial expression levels and model parameters, and
  - a phase that refines the expression estimates.
- Salmon builds a probabilistic model based on the sequencing data, which includes more information, and then improves the conditional probability that a fragment is part of a transcript.
- Important: the methods for transcript identification and quantification will report normalized quantification values as output and not counts of mapped reads.
- Therefore, the choice of the method for differential expression analysis should consider the type of data input expected (count or normalized values).

Salmon

- **What it does**
  Quantifies expression from RNA-seq reads given a transcriptome, outputting transcript-level (and gene-level via aggregation) TPMs and counts.
- **How it works (conceptually)**
  - Builds a **quasi-mapping** / lightweight index of the transcriptome.
  - Maps reads to transcripts quickly.
  - Uses advanced bias-correction models (GC bias, positional bias, sequence bias) and an Expectation–Maximization / Variational Bayes optimization step to refine abundances.
- **Key points**
  - **very fast**, often comparable to or faster than Kallisto.
  - Strong emphasis on **bias correction** (GC, sequence, positional, fragment length), which can improve accuracy in many datasets.
  - Can work:
    - **Directly from reads** (quasi-mapping mode), or
    - From **pre-computed genome alignments** (alignment-based mode).

Kallisto

- **What it does**
  Takes RNA-seq reads + a **transcriptome FASTA**, and estimates how many reads came from each transcript
- **How it works**
  - Builds a **k-mer index** of all transcripts.
  - For each read, finds where its k-mers occur in the transcriptome and determines the compatible transcripts (**pseudoalignment**—no full base-by-base alignment).
  - Uses an **EM algorithm** to assign reads probabilistically to transcripts and estimate abundances.
- **Key points**
  - Very **fast** and memory-efficient.
  - Great for **transcript-level quantification** when you have a good reference transcriptome.
  - Doesn't produce traditional genome BAMs (because it doesn't do full alignment).



Kallisto

- **Pros**
  - **Very fast & lightweight**
    Pseudoalignment with a k-mer index → super quick on typical RNA-seq (often minutes per sample on a laptop).
  - **Low memory use**
    Good for big transcriptomes on modest hardware.
  - **Simple workflow**
    Build index → quantify; outputs TPMs and estimated counts; nice for transcript-level work.
  - **Mature & well-tested**
    Widely used in the field, plenty of tutorials and wrappers.
- **Cons**
  - **Less rich bias correction**
    Has some bias handling, but generally considered less extensive than Salmon's (especially for sequence/GC/positional biases).
  - **Transcriptome-only**
    No native "alignment-based" mode; it works from reads + transcriptome, not from a genome BAM.
  - **Less built-in diagnostics**
    Compared with Salmon, fewer detailed QC / auxiliary outputs (e.g. mapping model details, bias estimates).
- **Good choice when:**
  You want **very fast, straightforward transcript quantification** from a known transcriptome and don't need advanced bias modeling.

Salmon

- **Pros**
  - **Very fast as well**
    Quasi-mapping is comparable to Kallisto in speed, often just as fast in practice.
  - **Strong bias correction**
    Models **GC**, **positional**, **sequence**, and **fragment-length** biases; this can noticeably improve accuracy for tricky libraries (e.g., strong GC bias).
  - **Flexible input modes**
    - Lightweight (quasi-mapping) mode from raw reads
    - **Alignment-based mode** from existing genome BAMs
  - **Rich outputs**
    Bootstraps / Gibbs samples, effective lengths, various auxiliary files that can feed into downstream tools (e.g. tximport, sleuth, DRIMSeq) and help assess uncertainty.
  - **Good integration**
    Widely supported in modern workflows (Snakemake/Nextflow pipelines, tximport in R, etc.).
- **Cons**
  - **More knobs & complexity**
    More options for bias models, library types, decoy sequences, etc.; easy to misconfigure if you don't read the docs.
  - **Slightly heavier footprint**
    Differences are small on typical systems, but it tends to feel a bit more "feature-heavy" than Kallisto.
- **Good choice when:**
  You want **state-of-the-art accuracy and bias correction**, or need to work from existing genome alignments as well as from raw reads.

Alignment output

- It contains all the mappings of a read and information such as alignment position and alignment score identified as MAPQ (acronym for MAPping Quality), which is presented in Phred
- File types: SAM and/or BAM

SAM

- A SAM (Sequence Alignment/Map) file is a tab-delimited text file that stores sequence reads aligned to a reference genome
- SAM file structure
  - Header: Starts with an "@" symbol and contains metadata about the reference sequences, the software used, and read groups.
  - Alignment: Each line after the header represents a single read alignment and includes 11 mandatory fields plus optional fields for more detailed information.
  - Mandatory fields: Include information like the read ID, reference sequence name, mapping position, and CIGAR string, which describes how the read aligns to the reference.
- CIGAR string: A sequence of operations (e.g., M for match, I for insertion, D for deletion) that describe the alignment.

```
@HD VN:1.5 SO:coordinate
@SQ SN:ref LN:45
r001   99 ref  7 30 8M2I4M1D3M = 37  39 TTAGATAAAGGATACTG *
r002    0 ref  9 30 3S6M1P1I4M * 0   0 AAAAGATAAGGATA    *
r003    0 ref  9 30 5S6M       * 0   0 GCCTAAGCTAA       * SA:Z:ref,29,-,6H5M,17,0;
r004    0 ref 16 30 6M14N5M    * 0   0 ATAGCTTCAGC       *
r003 2064 ref 29 17 6H5M       * 0   0 TAGGC             * SA:Z:ref,9,+,5S6M,30,1;
r001  147 ref 37 30 9M         = 7 -39 CAGCGGCAT         * NM:i:1
```

Header section

Alignment section

Optional fields in the format of TAG:TYPE:VALUE

QUAL: read quality; * meaning such information is not available

SEQ: read sequence

TLEN: the number of bases covered by the reads from the same fragment. Plus/minus means the current read is the leftmost/rightmost read. E.g. compare first and last lines.

PNEXT: Position of the primary alignment of the NEXT read in the template. Set as 0 when the information is unavailable. It corresponds to POS column.

RNEXT: reference sequence name of the primary alignment of the NEXT read. For paired-end sequencing, NEXT read is the paired read, corresponding to the RNAME column.

CIGAR: summary of alignment, e.g. insertion, deletion

MAPQ: mapping quality

POS: 1-based position

RNAME: reference sequence name, e.g. chromosome/transcript id

FLAG: indicates alignment information about the read, e.g. paired, aligned, etc.

QNAME: query template name, aka. read ID

https://bioinformaticamente.com/2021/03/03/sam-bam/

BAM file

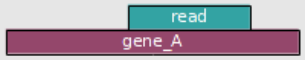
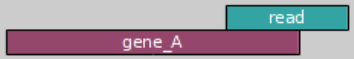
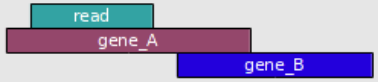
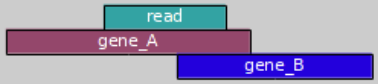
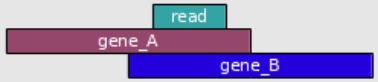
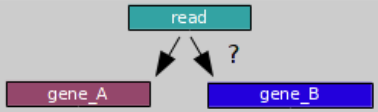
- It is a compressed binary file that stores the results of aligning DNA sequences to a reference genome.
- It is the binary, machine-readable version of the human-readable SAM file, making it more compact and efficient
- BAM files are used with an index file (BAI) for fast random access to specific regions

Step 3. Counting methods

- The approach in considering these mappings allows variations.
  - It is possible to consider only mappings with Phred values above a threshold or reads that obtained unique mapping.
  - Counting tools: process files in SAM and/or BAM format and produce a table with genes and the number of mapped reads.

Step 3. Counting

- It is necessary to consider the alignments options, including
  - read fully aligned to a gene;
  - read partially aligned to a gene;
  - read aligned to a junction (intron and exon);
  - read aligned to a junction of exons (no alignment with intron);
  - read partially aligned to two genes, and
  - read aligned to two genes.
- For this task some methods can be used together or alone, such as the
  - HTSeq-count
  - the BEDTools toolkit
  - featureCounts.

	union	intersection _strict	intersection _nonempty
	gene_A	gene_A	gene_A
	gene_A	no_feature	gene_A
	gene_A	no_feature	gene_A
	gene_A	gene_A	gene_A
	gene_A	gene_A	gene_A
	ambiguous (both genes with --nonunique all)	gene_A	gene_A
	ambiguous (both genes with --nonunique all)		
	alignment_not_unique (both genes with --nonunique all)		

Step 3. Counting

- The choice of method and how to consider mappings in the count should be made based on the dataset and its properties.
- Where little prior knowledge is available, a comparison is recommended between the count in the most restrictive mode and most permissive mode of each software tool to define an adequate parameterization.

HTSeq-count

- **What it is**
  Python-based command (part of the HTSeq package) that takes a BAM/SAM + GTF/GFF and outputs counts per feature (usually gene).
- **Pros**
  - **Simple, well-known interface**
    One command, one output file; very common in RNA-seq tutorials and older pipelines.
  - **Flexible counting modes**
    union, intersection-strict, intersection-nonempty for handling overlapping features.
  - **Good for teaching / small projects**
    Clear logic, easy to script around in Python if you use HTSeq as a library.
- **Cons**
  - **Slow and memory-heavier on large datasets**
    Single-threaded and Python-based; can be noticeably slower than featureCounts, especially for many samples or deep sequencing.
  - **Per-read processing overhead**
    Not ideal for huge cohorts or high-throughput pipelines.
  - **Mostly focused on annotation-based counting**
    Less natural for exotic region operations than a general toolkit like BEDTools.
- **Best use case:**
  Small–medium RNA-seq projects, teaching, or when you want transparent counting logic and don't mind speed.

BEDTools toolkit

- **What it is**
  A general **genomic intervals toolbox**. For counting reads, people commonly use coverage, intersect -c, or related commands on BAM/BED vs. annotation BED/GFF.
- **Pros**
  - **Very flexible**
    Can count coverage over *any* genomic regions (exons, peaks, promoters, windows, custom tiling, etc.), not just genes. Great for ChIP-seq, ATAC-seq, etc.
  - **Composability**
    You can chain multiple operations: merge regions, subtract blacklists, intersect multiple sets, then count.
  - **Mature & widely used**
    Very well tested, lots of documentation and examples for general interval logic.
- **Cons**
  - **Not RNA-seq specific**
    It doesn't know about exons vs genes, strand-specific counting, paired-end fragment logic, counting modes for overlapping genes, etc., unless you script that yourself.
  - **More work to mirror "gene-level" counting**
    To emulate what featureCounts/HTSeq-count do for genes, you often need multiple steps (exon union per gene, handling overlaps, strand filters, etc.).
  - **Speed is good but not specialized**
    For pure RNA-seq gene counting, featureCounts is usually faster and easier.
- **Best use case:**
  When you want **general genomic coverage/overlap operations**, custom regions, or non-RNA-seq applications (ChIP, ATAC, WGBS, etc.), and don't need RNA-seq-specific bells and whistles out of the box.

featureCounts

- **What it is**
  A highly optimized C program (part of the Subread suite) specifically for **counting reads/fragments** to genomic features (genes, exons, etc.).
- **Pros**
  - **Very fast and efficient**
    Multithreaded C implementation; often 5–10× faster than HTSeq-count on big datasets, with low memory use.
  - **Rich RNA-seq-specific options**
    - Strand specificity
    - Paired-end fragment counting vs. single-read
    - Multi-mapping handling
    - Exon-based or gene-based counting
  - **Handles large projects well**
    Designed for high-throughput; great choice for many samples or high depth.
  - **Natural integration**
    Widely used with edgeR/limma-voom, DESeq2, etc.; well documented.
- **Cons**
  - **Less general than BEDTools**
    Great for feature counting, but not a full interval-manipulation toolbox.
  - **Command-line options can be dense**
    Many flags; easy to mis-set strand/multi-overlap behaviour if you don't read the docs.
  - **Less transparent than HTSeq-count's simple Python logic**
    Harder to "peek inside" the algorithm unless you read the manual/implementation.
- **Best use case:**
  Standard **RNA-seq (or similar) read → gene/exon counting** at scale, where speed and robustness are important.